LIFE OF A PACKET

0️⃣ You type facebook.com and press Enter

At this point, nothing has gone on the wire yet.

The browser first answers:

- Is this HTTP or HTTPS? → modern browsers assume HTTPS
- Do I already know the IP?
- Do I have an open connection I can reuse?

1️⃣ URL parsing (browser logic)

Browser breaks the URL into parts:

Part	Value
Scheme	https
Hostname	facebook.com
Port	443 (default for HTTPS)
Path	/

If HTTPS → browser knows:

- TLS handshake is required
- TCP must be established first

2️⃣ Browser cache check (cheap wins first)

Before touching the network:

- DNS cache (browser-level)
- OS DNS cache
- Connection reuse (HTTP/2 / HTTP/3)

If IP is cached → skip DNS entirely
(huge performance win)

👉 As a network engineer, this is why:

nslookup facebook.com

might show a delay but browser loads instantly.

3️⃣ DNS resolution (this is where networking starts)

If IP not cached:

3.1 Stub resolver (your OS)

Your OS asks:

“Who is my DNS server?”

Usually:

- ISP DNS
- Corporate DNS
- Public DNS (8.8.8.8, 1.1.1.1)

3.2 Recursive DNS query (step-by-step)

Your DNS server does not magically know Facebook.

It walks the DNS tree:

Root DNS (.)

↓

.com TLD server

↓

facebook.com authoritative server

Each step returns:

- NS records
- A / AAAA records
- TTL values

Eventually:

facebook.com → 157.240.xxx.xxx

✓ Multiple IPs returned (load balancing, geo-DNS)
✓ TTL controls caching behavior

👉 Engineer mindset:

- Slow site? Check DNS latency
- Wrong site? DNS poisoning / hijack

4⃣ ARP (local network reality check)

Now your machine knows Facebook's IP — but...

"How do I actually send packets?"

4.1 Is the destination local?

- Compare destination IP with subnet mask
- Result: ❌ not local

So traffic must go to:
➡ Default Gateway

4.2 ARP request

If MAC of gateway unknown:

Who has 192.168.1.1? Tell 192.168.1.10

Gateway replies with MAC address.

Now your PC can build an Ethernet frame.

5️⃣ TCP 3-way handshake (transport layer)

Facebook uses TCP (still, even with HTTP/3 rising).

5.1 SYN

SRC: random high port (e.g. 52134)

DST: 443

Flags: SYN

5.2 SYN-ACK

Facebook server replies:

SYN + ACK

5.3 ACK

Client acknowledges.

🎉 TCP session established

👉 Things negotiated here:

- MSS
- Window scaling
- SACK

👉 Troubleshooting clues:

- SYN sent, no reply → firewall / routing issue
- SYN-ACK seen, ACK missing → client issue

6️⃣ TLS handshake (security layer)

Now encryption starts.

6.1 ClientHello

Browser sends:

- TLS version
- Cipher suites
- SNI = facebook.com (important!)
- ALPN (HTTP/1.1, HTTP/2, HTTP/3)

6.2 ServerHello

Facebook replies:

- Selected cipher
- Certificate chain
- Public key

6.3 Certificate validation

Browser checks:

- **Signed by trusted CA?**
- **Domain matches?**
- **Expired?**
- **Revoked?**

If any check fails → big red warning 😬

6.4 Key exchange

- **Session keys generated**
- **Symmetric encryption starts**

From now on:
🔒 Payload is encrypted
(you only see metadata on the wire)

7️⃣ HTTP request (finally!)

Now the browser sends:

GET / HTTP/2

Host: facebook.com

User-Agent: Chrome/144

Cookies: session, tracking

Over:

- **TLS**
- **TCP**
- **IP**
- **Ethernet/Wi-Fi**

8️⃣ Facebook's backend magic

Inside Facebook's network:

- **Load balancer (L4/L7)**
- **Geo-routing**
- **CDN (images, JS, CSS)**
- **Cache lookup**
- **App servers**
- **Databases**

Your request may hit:

- **Singapore POP**
- **Or nearest APAC DC**

👉 IP you connected to ≠ actual app server

9️⃣ HTTP response

Server replies:

- Status code: 200 OK
- HTML content
- Headers:
  - Cache-Control
  - Content-Security-Policy
  - Set-Cookie

Browser receives initial HTML only.

🔟 Page explosion (most people forget this)

HTML is parsed and triggers dozens / hundreds of new requests:

- CSS
- JS
- Images
- Fonts
- Ads
- Tracking pixels
- API calls

Modern browsers:

- Use HTTP/2 multiplexing
- Reuse same TCP connection
- Parallelize aggressively

This is where performance lives or dies.

1️⃣1️⃣ Rendering (client-side)

Browser pipeline:

1. Build DOM
2. Build CSSOM
3. Create render tree
4. Layout
5. Paint
6. Composite

JS can:

- Block rendering

- **Trigger reflows**
- **Make async API calls**

1️⃣ 2️⃣ Connection persistence

Connection stays open:

- **Keep-Alive**
- **Reused for future requests**
- **Closed after idle timeout**